

Les extensions de Redux

Table des matières

I. Contexte	3
II. React-Redux et ReduxToolkit	3
A. React-Redux et ReduxToolkit	3
B. Exercice : Quiz	8
III. Redux Persist	9
A. Redux Persist	9
B. Exercice : Quiz	14
IV. Essentiel	15
V. Auto-évaluation	15
A. Exercice	15
B. Test	15
Solutions des exercices	16

I. Contexte

Durée : 1 H

Environnement de travail : Navigateur Web - Visual Studio Code

Contexte

Dans le monde du développement web, la création d'applications complexes nécessite une gestion efficace de l'état de l'application. C'est là que React-Redux entre en jeu.

React-Redux est une bibliothèque qui combine les fonctionnalités de React, une bibliothèque JavaScript pour la création d'interfaces utilisateur, et Redux, un conteneur de gestion d'état prévisible pour les applications JavaScript. L'utilisation de React-Redux offre de nombreux avantages, notamment une gestion simplifiée de l'état de l'application, une architecture clairement définie et une facilité d'intégration avec React.

Dans ce cours, nous allons explorer les fonctionnalités essentielles de React-Redux, ainsi que les outils complémentaires tels que Redux Toolkit et Redux Persist. Nous allons découvrir comment les installer, les configurer et les utiliser pour optimiser le développement d'applications React.

II. React-Redux et ReduxToolkit

A. React-Redux et ReduxToolkit

Méthode Initialisation de l'application

Dans ce cours nous allons utiliser le bundler « Vite » pour initialiser notre application React, mais vous pouvez choisir d'utiliser des méthodes alternatives telles que CRA ou encore initialiser votre application « *from scratch* ».

Nous créons au préalable un dossier sur notre système, nous accédons à ce dossier depuis notre éditeur de code, puis dans le terminal de l'éditeur nous tapons la commande suivante pour initier une application React basée sur « Vite » :

```
1 npm create vite@latest
```

Vous pouvez également utiliser yarn ou pnpm pour initier notre application.

Nous suivons les différentes étapes requises pour initier le projet : indiquer le nom de l'application, choisir un environnement de développement React et l'utilisation de JavaScript

Enfin, nous pouvons supprimer le contenu du composant « *App.jsx* » qui contient les éléments de présentation de « Vite ».

Remarque Version de React

Veuillez noter que dans ce cours, nous utilisons la dernière version disponible de React « 18.02 ».

Présentation de React-Redux

React-Redux est une bibliothèque qui permet de combiner les fonctionnalités de React et Redux pour gérer l'état des applications React de manière efficace et prévisible. React, une bibliothèque JavaScript développée par Facebook, est utilisée pour la construction d'interfaces utilisateur réactives et dynamiques. Redux, quant à lui, est un conteneur de gestion d'état qui facilite le suivi et la mise à jour de l'état de l'application de manière prévisible.

React-Redux agit comme une couche d'abstraction entre React et Redux, offrant ainsi une intégration fluide et harmonieuse des deux bibliothèques. Il fournit des outils et des composants spécifiques pour faciliter l'utilisation de Redux avec React, en veillant à ce que les mises à jour de l'état soient correctement propagées aux composants React qui en dépendent.

L'un des principaux avantages de l'utilisation de React-Redux est la séparation claire entre la logique métier de l'application et la gestion de l'état. En isolant la logique de gestion d'état dans des composants spécifiques, React-Redux facilite la réutilisabilité du code, améliore la maintenabilité et favorise une architecture plus modulaire.

Présentation de Redux Toolkit

Redux Toolkit est une extension officielle de Redux qui simplifie et rationalise la façon dont vous pouvez gérer l'état de votre application. Il vise à résoudre certains des problèmes courants auxquels les développeurs sont confrontés lorsqu'ils utilisent Redux, tels que la configuration complexe, la duplication de code et la verbosité excessive.

Redux Toolkit fournit un ensemble d'outils et de conventions pour vous aider à écrire du code Redux de manière plus concise, lisible et maintenable. Il inclut notamment des fonctionnalités telles que la création automatique de tranches Redux, la gestion simplifiée des actions et des réducteurs, ainsi que des utilitaires pour l'immuabilité des données.

L'une des principales fonctionnalités de Redux Toolkit est la création de tranches Redux à l'aide de la fonction "createSlice". Cela vous permet de définir une tranche de l'état de l'application, comprenant à la fois son réducteur correspondant et les actions nécessaires pour le mettre à jour. Avec Redux Toolkit, la configuration initiale et la gestion des actions sont grandement simplifiées, vous permettant ainsi de vous concentrer sur la logique métier plutôt que sur les détails de mise en place.

En utilisant Redux Toolkit avec React-Redux, vous pouvez exploiter la puissance de Redux de manière plus intuitive et efficace. Cela simplifie le processus de création et de gestion de l'état de votre application React, tout en réduisant la quantité de code nécessaire.

Méthode Installation de React-Redux et Redux Toolkit

Nous pouvons maintenant installer les extensions React-Redux et Redux Toolkit dans notre application. Pour cela, accédez au dossier contenant notre application depuis le terminal et tapez la commande suivante :

```
1 npm install @reduxjs/toolkit react-redux
```

Remarque Versions des extensions

Veuillez noter que dans ce cours, nous utiliserons les dernières versions disponibles de React Redux et Redux Toolkit, qui sont respectivement la version « 8.0.5 » et « 1.9.5 ».

Méthode Introduire Redux dans l'architecture de l'application

Pour garder une architecture cohérente dans notre application, nous pouvons créer un nouveau répertoire dans le dossier « src » que nous allons nommer « redux », c'est dans ce dossier que nous allons injecter la logique principale de notre gestionnaire d'état.

Méthode Création du store avec Redux Toolkit

Au sein de notre nouveau dossier « redux », nous pouvons créer un nouveau fichier « store.js » dans lequel nous allons initier notre store en utilisant Redux Toolkit :

```
1 import { configureStore } from '@reduxjs/toolkit'
2 export default configureStore({
3   reducer : {
4
5   }
6 })
```

En premier lieu, nous avons importé la méthode « configureStore » de Redux Toolkit, puis nous avons utilisé cette méthode pour créer notre store.

Veuillez noter que nous passons un objet à cette méthode, qui contient une propriété « *reducer* », cette propriété va contenir l'ensemble des reducers que nous créerons dans application.

Méthode Fournir notre store à React

Pour transmettre le store à notre application, nous pouvons utiliser l'extension React Redux, qui va utiliser un « *Provider* » et transmettre notre store depuis la racine de notre application.

Dans le fichier « *main.jsx* » nous pouvons inscrire le code suivant :

```
1 import ReactDOM from 'react-dom/client'
2 import App from './App.jsx'
3 import './index.css'
4 import store from './redux/store.js'
5 import { Provider } from 'react-redux'
6
7 ReactDOM.createRoot(document.getElementById('root')).render(
8   <Provider store={store}>
9     <App />
10  </Provider>,
11 )
```

Ici, nous avons importé notre store créé précédemment, ainsi que « *Provider* » depuis l'extension React Redux, enfin nous avons englobé l'ensemble de notre application dans le « *Provider* » en lui passant notre store en tant qu'attribut.

Introduction au Slice de Redux Toolkit

Dans Redux Toolkit, un Slice est une entité centrale qui regroupe un ensemble d'actions et un réducteur spécifiques pour gérer une partie de l'état de votre application. Il fournit une abstraction pratique pour organiser et gérer les différentes parties de votre état global.

L'utilisation de Slices avec Redux Toolkit simplifie considérablement le flux de travail de développement avec Redux.

Méthode Création d'un nouveau fichier contenant notre slice

Nous pouvons mettre en place un nouveau fichier qui va contenir notre premier slice.

Dans le dossier « *redux* », créez un fichier nommé « *counterSlice.js* », et nous pouvons y inscrire le code suivant :

```
1 import { createSlice } from "@reduxjs/toolkit"
2 export const counterSlice = createSlice({
3   name : 'counter',
4   initialState : {
5     value : 0
6   },
7   reducers : {
8     increment : (state) => {
9       state.value++
10     },
11     decrement : (state) => {
12       state.value--
13     }
14   }
15 })
16 export const { increment, decrement } = counterSlice.actions
17 export default counterSlice.reducer
```

Ici, nous avons importé la fonction “createSlice” de Redux Toolkit et utilisé cette fonction pour créer un slice.

Nous avons défini plusieurs propriétés dans notre « *counterSlice* » :

Premièrement, nous avons spécifié un nom à notre slice qui porte la valeur ‘counter’, ensuite nous avons passé l’état initial du compteur, qui est un objet qui possède une valeur de 0.

Nous avons également défini « *increment* » et « *decrement* » qui vont respectivement incrémenter et décrémenter notre état. Notez que Redux Toolkit nous permet d’écrire une logique mutative dans les reducers. Cependant , il ne mute pas réellement l’état, car il utilise la bibliothèque Immer en interne.

Enfin, nous exportons nos différents éléments afin de pouvoir les utiliser dans d’autres parties de l’application.

Remarque Immer

Immer est une bibliothèque qui permet de manipuler l’immuabilité de manière plus facile et intuitive, en utilisant une syntaxe proche de la manipulation mutable des objets. Elle offre une expérience de développement plus agréable en évitant la nécessité de copier et de créer des objets immutables à chaque mise à jour de l’état.

Avec l’intégration de Immer dans Redux Toolkit, vous pouvez utiliser la fonction createSlice pour définir des tranches de réducteurs (reducers) qui utilisent Immer sous le capot.

Méthode Ajouter le reducer à notre store

Nous allons maintenant transmettre notre reducer à notre store dans le fichier « *store.js* » :

```
1 import counterReducer from './counterSlice'
2 export default configureStore({
3   reducer : {
4     counter : counterReducer
5   }
6 })
```

Nous avons importé notre « *counterReducer* » dans notre store et nous l’avons transmis à notre store.

Méthode Création de notre composant counter

Nous allons maintenant créer notre composant « *Counter.jsx* » dans le répertoire « *src* », ce composant nous permettra d’afficher la valeur du compteur dans l’application ainsi que de gérer les événements pour déclencher les actions d’incrémentation et de décrémentation. Nous pouvons écrire le code suivant :

```
1 export default function Counter(){
2   return(
3     <div>
4       <span>Valeur du compteur</span>
5       <button>Incrementé le compteur</button>
6       <button>Decrementé le compteur</button>
7     </div>
8   )
9 }
```

Nous avons créé un composant fonctionnel standard qui comprend un « *span* » dans lequel nous allons afficher la valeur de notre compteur, ainsi que deux boutons qui vont permettre de modifier la valeur du compteur. Maintenant, nous pouvons instancier ce composant dans le composant principal de notre application « *App.jsx* » via le code suivant :

```
1 import Counter from './Counter.jsx'
2 function App() {
3   return (
4     <Counter/>
5   )
6 }
7 export default App
```

Les Hooks de React-Redux

Les hooks de React Redux sont des fonctions fournies par la bibliothèque React Redux qui permettent de connecter facilement les composants React au store de Redux et d'accéder aux données de l'état et aux actions du store. Ces hooks simplifient la gestion de l'état global dans une application React en fournissant une interface plus concise et fonctionnelle.

Voici les hooks les plus couramment utilisés dans React Redux :

- **useSelector** : ce hook permet de sélectionner des données spécifiques à partir du store Redux. Il prend en argument une fonction de sélecteur qui spécifie les données que vous souhaitez extraire de l'état global. Le hook renvoie alors les données sélectionnées, et chaque fois que ces données sont mises à jour dans le store, le composant est automatiquement réexécuté pour refléter ces changements.
- **useDispatch** : ce hook renvoie une fonction de dispatch qui permet de déclencher des actions dans le store Redux. En utilisant ce hook, vous pouvez appeler la fonction de dispatch directement dans votre composant pour envoyer des actions au store et mettre à jour l'état global.
- **useStore** : ce hook permet d'accéder à l'instance du store Redux. Il renvoie l'objet complet du store, vous permettant ainsi d'accéder à toutes ses fonctionnalités telles que l'abonnement aux mises à jour de l'état, l'accès aux middlewares, etc. Cependant, l'utilisation de ce hook est généralement moins courante, car il est préférable de privilégier l'utilisation des autres hooks plus spécialisés.

Ces hooks offrent une alternative plus légère et plus simple à l'utilisation des composants connectés de React Redux.

Ils permettent de réduire la quantité de code nécessaire pour connecter un composant à un store Redux, en évitant le besoin de créer des composants connectés séparés. Les hooks offrent également une meilleure intégration avec les fonctionnalités de React, telles que l'utilisation des hooks de cycle de vie.

Méthode Accès au state avec useSelector

Nous allons maintenant utiliser le hook « *useSelector* » de React Redux afin d'accéder à notre état contenu dans le store, et ce, en vue d'afficher la valeur du compteur dans notre application. Pour cela, nous pouvons inscrire le code suivant dans notre composant « *Counter.jsx* » :

```
1 import { useSelector } from "react-redux"
2 export default function Counter(){
3   const count = useSelector(state => state.counter.value)
4   return(
5     <div>
6       <span>{count}</span>
7       <button>Incrementé le compteur</button>
8       <button>Décrementé le compteur</button>
9     </div>
10  )
```

```
11 }
```

Premièrement, nous avons importé le hook « *useSelector* » de React Redux, puis nous avons instancié une variable *count*, qui utilise ce hook pour accéder à la valeur de notre compteur contenue dans l'état du store.

Enfin, nous utilisons cette variable dans notre JSX afin d'afficher cette valeur dans l'application.

Méthode Dispatch d'une action avec useDispatch

Pour permettre à nos boutons de modifier la valeur de notre compteur, nous devons utiliser le hook « *useDispatch* » pour dispatcher les actions vers notre reducer :

```
1 import { useSelector, useDispatch } from "react-redux"
2 import { increment, decrement } from "../redux/counterSlice.js"
3
4 export default function Counter(){
5
6     const count = useSelector(state => state.counter.value)
7
8     const dispatch = useDispatch()
9
10    return(
11        <div>
12            <span>{count}</span>
13            <button onClick={() => dispatch(increment())}>
14                Incrémenter le compteur
15            </button>
16            <button onClick={() => dispatch(decrement())}>
17                Décrémenter le compteur
18            </button>
19        </div>
20    )
21 }
```

Ici, nous importons le hook « *useDispatch* » de React Redux, qui nous permet de dispatcher les actions « *increment* » et « *decrement* » que nous avons importées depuis notre slice.

Lors du clic sur l'un des boutons, le hook « *useDispatch* » dispatche l'action passée dans la fonction de notre attribut « *onClick* ».

Nous pouvons désormais voir la valeur du compteur être mise à jour dynamiquement dans notre application.

B. Exercice : Quiz

[solution n°1 p.17]

Question 1

Quelle bibliothèque faut-il utiliser pour intégrer Redux avec React ?

- ☐ Redux Toolkit
- ☐ React Router
- ☐ React Redux

Question 2

Redux Toolkit apporte une syntaxe plus concise et simplifiée par rapport à Redux seul ?

- ☐ Vrai
- ☐ Faux

Question 3

Quelle fonction de React Redux est utilisée pour connecter un composant React au store Redux ?

- ☐ connect
- ☐ useSelector
- ☐ createSelector

Question 4

Quelle est la méthode recommandée pour créer un slice dans Redux Toolkit ?

- ☐ createReducer
- ☐ configureSlice
- ☐ createSlice

Question 5

Quelle est la méthode pour créer un store Redux avec Redux Toolkit ?

- ☐ createStore
- ☐ configureStore
- ☐ createNewStore

III. Redux Persist

A. Redux Persist

Qu'est-ce que Redux Persist ?

Redux Persist est une bibliothèque JavaScript utilisée en conjonction avec Redux pour la gestion de l'état dans les applications React. L'un des principaux défis lors du développement d'applications React est de maintenir l'état de l'application de manière persistante, même lorsque la page est rechargée ou que l'application est redémarrée. C'est là que Redux Persist intervient.

Redux Persist permet de sauvegarder l'état de votre application Redux dans le stockage local (localStorage) ou le stockage de session (sessionStorage) du navigateur. Cela signifie que les données sont conservées même après la fermeture de l'application ou lorsque l'utilisateur recharge la page.

Pourquoi utiliser Redux Persist ?

La persistance de l'état est particulièrement utile pour des cas d'utilisation tels que la conservation de l'état d'authentification de l'utilisateur, les préférences utilisateur ou tout autre donnée que vous souhaitez conserver entre les sessions.

L'un des avantages clés de Redux Persist est sa simplicité d'utilisation. En utilisant Redux Persist, vous pouvez éviter d'écrire manuellement du code complexe pour gérer la persistance de l'état. La bibliothèque s'intègre facilement dans votre flux Redux existant et fournit des méthodes simples pour configurer et gérer la persistance.

Remarque	Version de Redux Persist
----------	--------------------------

Dans ce cours, nous allons nous baser sur la dernière version disponible de Redux Persist, la version 6.0.0.
--

Méthode Installation de Redux Persist

Pour installer Redux Persist dans nos dépendances, nous pouvons accéder au dossier de notre application depuis le terminal de notre éditeur de code et taper la commande suivante :

```
1 npm install redux-persist
```

Méthode Création d'un nouveau slice

Nous allons créer un nouveau slice qui va gérer la couleur de fond de notre application, pour cela nous créons un nouveau fichier « *colorSlice.js* » dans le dossier « *redux* » qui va contenir le code suivant :

```
1 import { createSlice } from "@reduxjs/toolkit"
2
3 export const colorSlice = createSlice({
4   name : 'color',
5   initialState : {
6     value : 'rgb(29,29,29)'
7   },
8   reducers : {
9     randomColor : (state) => {
10       let randomColor = `rgb(${Math.random() * 255},${Math.random() *
11 255},${Math.random() * 255})`
12       state.value = randomColor
13     }
14   })
15
16 export const { randomColor } = colorSlice.actions
17
18 export default colorSlice.reducer
```

Nous pouvons maintenant combiner nos deux reducers grâce à la méthode « *combineReducers* » fournie par Redux Toolkit, dans le fichier « *store.js* » nous inscrivons le code suivant :

```
1 import { configureStore } from '@reduxjs/toolkit'
2 import counterReducer from './counterSlice.js'
3 import colorReducer from './colorSlice.js'
4 import { combineReducers } from '@reduxjs/toolkit'
5
6 const rootReducer = combineReducers({
7   counter : counterReducer,
8   color : colorReducer
9 })
10
11 const store = configureStore({
12   reducer : rootReducer
13 })
14
15 export default store
```

Nous avons combiné nos deux reducers et avons transmis le résultat de cette combinaison à notre store.

Méthode Attribution de la couleur à notre application

Nous pouvons maintenant accéder à notre couleur contenue dans le store et l'attribuer à la couleur de fond de notre application. Pour cela, dans notre fichier « *App.jsx* », nous allons utiliser les hooks « *useSelector* » et « *useDispatch* ». Ainsi nous pourrons mettre à jour la couleur en dispatchant l'action « *randomColor* » de notre slice « *ColorSlice.js* » :

```
1 import { useEffect } from 'react'
2 import { useSelector, useDispatch } from 'react-redux'
3 import { randomColor } from './redux/colorSlice'
4 import Counter from './Counter'
5
6 export default function App() {
7
8   const color = useSelector(state => state.color.value)
9   const dispatch = useDispatch()
10
11   useEffect(()=>{
12     document.querySelector('body').style.backgroundColor = color
13   },[color])
14
15   return (
16     <>
17       <button onClick={() => { dispatch(randomColor()) }}>Change Background Color</button>
18       <Counter/>
19     </>
20   )
21 }
```

Notez que nous utilisons le hook « *useEffect* » de React, afin d'attribuer la nouvelle couleur au body de l'application à chaque fois que notre variable "color" est mise à jour.

Méthode Création d'un persistReducer

La première étape pour implémenter Redux Persist dans notre projet est de fournir un « *persistReducer* » à notre application auquel nous allons fournir deux paramètres. Le premier sera un objet de configuration auquel nous devons passer une clé d'identification, ainsi qu'un storage qui permettra de stocker le store persistant. Dans le cas d'application web, il sera stocké dans le *localStorage* du navigateur par défaut. Le second paramètre va faire référence aux reducers de notre application. Dans notre cas, nous allons fournir notre « *rootReducer* ». Ecrivez le code suivant dans le fichier « *store.js* » à la place des déclarations de store actuel :

```
1 import { persistReducer } from 'redux-persist'
2 import storage from 'redux-persist/lib/storage'
3
4 const persistConfig = {
5   key : 'root',
6   storage : storage
7 }
8
9 const persistedReducer = persistReducer(persistConfig,rootReducer)
10
11 const store = configureStore({
12   reducer : persistedReducer
13 })
14
15 export { store }
```

Notez que nous avons transmis le *persistedReducer* en tant que reducer pour notre store.

Méthode Utilisation de persistStore pour faire persister le store

Afin de faire persister le store que nous avons créé, nous devons utiliser « *persistStore* » de Redux Persist et lui transmettre notre store :

```
1 import { persistStore, persistReducer } from 'redux-persist'
2
3 const persistor = persistStore(store)
4
5 export { store, persistor }
```

Nous exportons également notre « *persistor* » afin de pouvoir le transmettre à notre application depuis sa racine.

Méthode Récupération de l'état persistant lors du rechargement de l'application

Nous devons récupérer l'état persistant créé précédemment à la racine de notre application afin que nous puissions y accéder dès le chargement de l'application. Pour cela nous devons utiliser « *persistGate* » de Redux Persist. Inscrivez le code suivant dans notre fichier « *main.js* » :

```
1 import { Provider } from 'react-redux'
2 import React from 'react'
3 import ReactDOM from 'react-dom/client'
4 import App from './App.jsx'
5 import './index.css'
6 import { PersistGate } from 'redux-persist/integration/react'
7 import { persistor, store } from '../src/redux/store.js'
8
9
10 ReactDOM.createRoot(document.getElementById('root')).render(
11   <Provider store={store}>
12     <PersistGate loading={null} persistor={persistor}>
13       <App />
14     </PersistGate>
15   </Provider>,
16 )
```

Nous pouvons spécifier « *null* » à l'attribut « *loading* » de notre « *PersistGate* », nous pouvons spécifier un composant particulier ou tout autre élément, qui s'affiche avant le chargement de la page et de son état persistant.

Remarque Redux DevTool

Si nous ouvrons Redux DevTool, nous pouvons remarquer que nous avons une visibilité sur l'état persistant présent dans notre application.

Par ailleurs, désormais si nous actualisons la page, nous conservons les valeurs respectives présentes dans l'état de notre compteur ou de la couleur.

Méthode Inclusion ou exclusion de certains états

Il est possible d'inclure et/ou exclure certains états de notre store persistant, pour cela nous pouvons spécifier des propriétés supplémentaires dans l'objet de configuration que nous fournissons à notre « *persistedReducer* » dans le fichier « *store.js* » :

```
1 const persistConfig = {
2   key : 'root',
3   storage : storage,
4   blacklist : ['counter']
5 }
```

En utilisant la propriété « *blacklist* » et en lui fournissant notre sous-ensemble d'état « *counter* », nous avons exclu le compteur de notre store persistant, de cette manière lorsque l'utilisateur recharge à nouveau notre application, la valeur du compteur n'est pas sauvegardée et est réinitialisée à son état initial.

Cette fonctionnalité peut être utile dans le cas où nous souhaitons faire le tri parmi les différents sous-ensembles d'état et sélectionner ceux qui ont besoin d'être sauvegardés ou non.

Dans la même logique, nous pouvons également spécifier la propriété « *whitelist* » qui permet d'inclure exclusivement certains sous-ensembles d'état.

Sérialiseur/désérialiseur

Les sérialiseurs et les désérialiseurs sont des fonctions utilisées pour convertir les données de l'état de Redux en une forme sérialisée (par exemple, une chaîne de caractères JSON) et vice versa.

Ils sont utilisés dans le cadre de la persistance des données, ce qui permet de conserver l'état de l'application entre les sessions ou les rafraîchissements de page.

Un sérialiseur est une fonction qui prend en entrée l'état de l'application et renvoie une représentation sérialisée de cet état.

Le désérialiseur, quant à lui, est une fonction qui prend en entrée la représentation sérialisée de l'état de l'application et renvoie l'état désérialisé correspondant. Le désérialiseur permet de reconstituer l'état de l'application à partir de sa forme sérialisée.

Explication de l'erreur générée

En effet, vous l'aviez peut-être remarqué, nous avons actuellement une erreur générée dans la console, cela est dû au fait que Redux n'accepte pas que des données non sérialisables, soit présentes dans l'état de l'application.

Cependant, Redux Persist utilise des actions telles que « *FLUSH* », « *REHYDRATE* », « *PAUSE* », « *PERSIST* », « *PURGE* » et « *REGISTER* » qui sont liées à son fonctionnement interne et que ne sont pas destinées à être transmises ou enregistrées de manière autonome.

Pour éviter les erreurs lors de la sérialisation de l'état de l'application, Redux Persist propose des mécanismes spécifiques, tels que les sérialiseurs et les désérialiseurs mentionnés précédemment, qui permettent de gérer la conversion de l'état de l'application de manière sérialisable. Les actions spéciales de Redux Persist ne sont pas incluses dans cette sérialisation, car elles ne font pas partie de l'état de l'application lui-même.

Méthode Gestion de l'erreur

Nous pouvons résoudre ce problème en configurant le middleware de notre store en utilisant la fonction « *getDefaultMiddleware* » fournie par Redux Toolkit.

Cette fonction retourne une liste de middleware pour la gestion des actions asynchrones.

Il est possible de passer un objet de configuration supplémentaire afin de spécifier les actions à ignorer lors de la vérification de sérialisabilité, nous pouvons inscrire le code suivant dans notre fichier « *store.js* » :

```
1 import {
2   persistStore, persistReducer, FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE, REGISTER
3 } from 'redux-persist'
4
5 const store = configureStore({
6   reducer: persistedReducer,
7   middleware: (getDefaultMiddleware) =>
8     getDefaultMiddleware({
9       serializableCheck: {
10         ignoredActions: [FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE, REGISTER],
11       },
12     })
13 })
```

```
12   } ),  
13 }
```

En ajoutant la liste « *ignoredActions* », nous indiquons à Redux Toolkit de ne pas effectuer la vérification de sérialisabilité pour ces actions spécifiques. Ainsi l'erreur générée n'est plus présente.

B. Exercice : Quiz

[solution n°2 p.18]

Question 1

Quel est le rôle principal de Redux Persist ?

- ☐ Gérer l'état global d'une application React
- ☐ Persister l'état de l'application dans le stockage local
- ☐ Créer des slices personnalisés

Question 2

Quel est le stockage utilisé par défaut par Redux Persist ?

- ☐ IndexedDB
- ☐ LocalStorage
- ☐ WebSQL

Question 3

Quelle méthode de Redux Persist est utilisée pour créer un persistor ?

- ☐ persistStore
- ☐ createPersistor
- ☐ createStoreWithPersistor

Question 4

Quelle option de configuration de Redux Persist permet de définir les clés d'exclusion pour la persistance ?

- ☐ whitelist
- ☐ blacklist
- ☐ excludeKeys

Question 5

Quelle fonction de Redux Toolkit est utilisée pour configurer le middleware du store ?

- ☐ getDefaultMiddleware
- ☐ persistStore
- ☐ persistReducer

IV. Essentiel

Dans la première partie de ce cours, nous avons exploré en détail React-Redux et Redux Toolkit, deux outils essentiels pour la gestion de l'état dans nos applications React. Nous avons commencé par comprendre l'importance des avantages offerts par Redux Toolkit, qui simplifie considérablement le processus de configuration et de gestion de Redux dans nos projets.

Nous avons également vu comment accéder à l'état du store en utilisant useSelector et à dispatcher des actions en utilisant useDispatch.

Puis, nous avons introduit Redux Persist, un outil puissant pour persister l'état de notre application Redux et les avantages qu'il offre.

Enfin, nous avons abordé des notions importantes comme l'inclusion ou l'exclusion de certains états dans la persistance des données.

Grâce à ce cours, vous avez acquis une solide compréhension de React-Redux, Redux Toolkit et Redux Persist, ainsi que des compétences pratiques pour les utiliser dans vos projets React.

V. Auto-évaluation

A. Exercice

Imaginez avoir besoin d'une nouvelle fonctionnalité dans notre application qui permet à l'utilisateur de saisir son nom et de l'afficher sur la page d'accueil. Nous souhaitons que lorsque l'utilisateur recharge l'application, son nom soit sauvegardé et affiché dans notre application.

Question 1

[solution n°3 p.19]

Créer un nouveau sous-ensemble d'état pour notre store.

Question 2

[solution n°4 p.19]

Récupérer la valeur du nom de l'utilisateur dans notre store, afficher cette valeur dans l'application et gérer la modification de l'état en fonction du nom soumis par l'utilisateur.

B. Test

Exercice 1 : Quiz

[solution n°5 p.20]

Question 1

Quel hook de React Redux est utilisé pour dispatcher des actions vers le store Redux ?

- ☐ dispatch
- ☐ useDispatch
- ☐ createDispatch

Question 2

Quelle est la méthode recommandée pour mettre à jour l'état d'un slice dans Redux Toolkit ?

- ☐ En modifiant directement l'état dans le reducer du slice
- ☐ En dispatchant une action avec dispatch et en passant les nouvelles données en tant que payload
- ☐ En utilisant le hook useSlice

Question 3

Quelle méthode de Redux Toolkit permet de combiner plusieurs reducers en un seul reducer racine ?

- ☐ combineReducers
- ☐ configureStore
- ☐ createSlice

Question 4

Comment peut-on inclure certains reducers de la persistance avec Redux Persist ?

- ☐ En utilisant l'option includeReducers lors de la configuration
- ☐ En utilisant l'option purge lors de la configuration
- ☐ En utilisant l'option whitelist lors de la configuration

Question 5

Quelle commande devez-vous taper dans le terminal pour installer Redux Toolkit ?

- ☐ npm install react-toolkit
- ☐ npm install @reduxjs/toolkit
- ☐ npm install redux

Solutions des exercices

Exercice p. 8 Solution n°1**Question 1**

Quelle bibliothèque faut-il utiliser pour intégrer Redux avec React ?

- ☐ Redux Toolkit
- ☐ React Router
- ☒ React Redux

 La bibliothèque à utiliser pour intégrer Redux avec React est React Redux. React Redux est une liaison officielle entre Redux et React. Elle fournit des composants et des utilitaires pour faciliter l'intégration de Redux avec React et permet de connecter les composants React à l'état géré par Redux.

Question 2

Redux Toolkit apporte une syntaxe plus concise et simplifiée par rapport à Redux seul ?

- ☒ Vrai
- ☐ Faux

 L'un des principaux avantages de Redux Toolkit par rapport à Redux seul est qu'il introduit une syntaxe plus concise et simplifiée pour la définition des actions, des réducteurs et des magasins.

Question 3

Quelle fonction de React Redux est utilisée pour connecter un composant React au store Redux ?

- ☐ connect
- ☒ useSelector
- ☐ createSelector

 La fonction useSelector est un hook fourni par React Redux qui permet à un composant React de sélectionner une partie spécifique de l'état du store Redux. Elle permet au composant de s'abonner aux mises à jour de cet état et de récupérer les données nécessaires.

Question 4

Quelle est la méthode recommandée pour créer un slice dans Redux Toolkit ?

- ☐ createReducer
- ☐ configureSlice
- ☒ createSlice

 « *createSlice* » est un utilitaire fourni par Redux Toolkit qui permet de définir à la fois les réducteurs et les actions d'un slice en une seule étape.

Lorsque vous utilisez « *createSlice* », vous fournissez un nom pour le slice, un état initial, et un objet qui définit les réducteurs pour gérer les différentes actions.

Question 5

Quelle est la méthode pour créer un store Redux avec Redux Toolkit ?

- ☐ createStore
- ☒ configureStore
- ☐ createNewStore

Q « *configureStore* » est une fonction fournie par Redux Toolkit qui permet de créer un store Redux avec diverses fonctionnalités et configurations prédéfinies.

Lorsque vous utilisez « *configureStore* », vous pouvez spécifier plusieurs options telles que les réducteurs, les middleware, les extensions Redux DevTools, et d'autres configurations liées à la gestion de l'état.

Exercice p. 14 Solution n°2

Question 1

Quel est le rôle principal de Redux Persist ?

- ☐ Gérer l'état global d'une application React
- ☒ Persister l'état de l'application dans le stockage local
- ☐ Créer des slices personnalisés

Q Le rôle principal de Redux Persist est de persister l'état de l'application dans le stockage local. Redux Persist est une bibliothèque qui fonctionne avec Redux et permet de sauvegarder et de restaurer l'état de l'application même après un rechargement de la page ou une fermeture de l'application.

Question 2

Quel est le stockage utilisé par défaut par Redux Persist ?

- ☐ IndexedDB
- ☒ LocalStorage
- ☐ WebSQL

Q Le stockage utilisé par défaut par Redux Persist est LocalStorage. Redux Persist utilise le stockage local du navigateur, qui est accessible via l'API JavaScript localStorage. Le stockage local (LocalStorage) est un mécanisme de stockage persistant dans le navigateur qui permet de stocker des données sous forme de paires clé-valeur.

Question 3

Quelle méthode de Redux Persist est utilisée pour créer un persistor ?

- ☒ persistStore
- ☐ createPersistor
- ☐ createStoreWithPersistor

Q La méthode utilisée pour créer un persistor dans Redux Persist est persistStore. La fonction persistStore est fournie par Redux Persist et permet de créer un persistor qui gère la persistance de l'état de l'application.

Question 4

Quelle option de configuration de Redux Persist permet de définir les clés d'exclusion pour la persistance ?

- ☐ whitelist
 - ☒ blacklist
 - ☐ excludeKeys
- Q** L'option de configuration de Redux Persist qui permet de définir les clés d'exclusion pour la persistance est blacklist. Lorsque vous configurez Redux Persist, vous pouvez spécifier cette option pour indiquer quelles clés de l'état ne doivent pas être persistées.

Question 5

Quelle fonction de Redux Toolkit est utilisée pour configurer le middleware du store ?

- ☒ getDefaultMiddleware
 - ☐ persistStore
 - ☐ persistReducer
- Q** La fonction de Redux Toolkit utilisée pour configurer le middleware du store est getDefaultMiddleware. Il s'agit d'une fonction fournie par Redux Toolkit qui renvoie le middleware par défaut utilisé par Redux Toolkit.

p. 15 Solution n°3

Nous pouvons créer un nouveau fichier "userSlice.js" qui va contenir le code suivant :

```
1 import { createSlice } from "@reduxjs/toolkit"
2
3 export const userSlice = createSlice({
4   name : 'userName',
5   initialState : {
6     value : 'Utilisateur inconnu'
7   },
8   reducers : {
9     setUsername : (state, action) => {
10       console.log(state, action)
11       state.value = action.payload
12     }
13   }
14 })
15
16 export const { setUsername } = userSlice.actions
17
18 export default userSlice.reducer
```

Nous avons créé un nouveau slice qui porte le nom « *useSlice* », nous pouvons maintenant joindre le reducer de notre slice au store de notre application, dans le fichier « *store.js* » :

```
1 import userReducer from './userSlice.js'
2
3 const rootReducer = combineReducers({
4   counter : counterReducer,
5   color : colorReducer,
6   user : userReducer
7 })
```

p. 15 Solution n°4

Nous pouvons créer un nouveau composant « *User.jsx* » et y inscrire notre logique pour récupérer la valeur contenue dans le store et gérer la modification de cet état :

```

1 import { useDispatch, useSelector } from "react-redux"
2 import { setUsername } from "../redux/userSlice.js"
3
4 export default function User(){
5
6     const userName = useSelector(state => state.user.value)
7     const dispatch = useDispatch()
8
9     const handleSubmit = (e)=>{
10         e.preventDefault()
11         console.log(e.target.elements.userName.value)
12         dispatch(setUsername(e.target.elements.userName.value))
13     }
14
15     return(
16         <div>
17             <span>{userName}</span>
18             <form onSubmit={handleSubmit}>
19                 <input type="text" name="userName"/>
20                 <button type="submit">Soumettre</button>
21             </form>
22
23         </div>
24     )
25 }

```

Actuellement, dans la configuration de notre « *persistReducer* », nous avons spécifié l'exclusion de notre état « *counter* », par défaut les autres sous-ensembles d'état seront automatiquement persistants, nous n'avons donc pas besoin de réaliser de modifications particulières afin de sauvegarder le nom de l'utilisateur.

Exercice p. 15 Solution n°5

Question 1

Quel hook de React Redux est utilisé pour dispatcher des actions vers le store Redux ?

- ☐ dispatch
- ☒ useDispatch
- ☐ createDispatch

 Le hook de React Redux utilisé pour dispatcher des actions vers le store Redux est useDispatch. Ce hook est fourni par React Redux et permet d'accéder à la fonction dispatch du store Redux.

Question 2

Quelle est la méthode recommandée pour mettre à jour l'état d'un slice dans Redux Toolkit ?

- ☐ En modifiant directement l'état dans le reducer du slice
- ☒ En dispatchant une action avec dispatch et en passant les nouvelles données en tant que payload
- ☐ En utilisant le hook useSlice

 La méthode recommandée pour mettre à jour l'état d'un slice dans Redux Toolkit est en dispatchant une action avec dispatch et en passant les nouvelles données en tant que payload.

-  Dans Redux Toolkit, vous définissez les réducteurs pour gérer les actions d'un slice à l'aide de `createSlice`. Lorsque vous souhaitez mettre à jour l'état d'un slice, vous dispatchez une action correspondante en utilisant `dispatch`. Vous pouvez passer les nouvelles données à mettre à jour en tant que payload de l'action.

Question 3

Quelle méthode de Redux Toolkit permet de combiner plusieurs reducers en un seul reducer racine ?

- ☒ `combineReducers`
- ☐ `configureStore`
- ☐ `createSlice`

-  La méthode de Redux Toolkit qui permet de combiner plusieurs reducers en un seul reducer racine est `combineReducers`. `combineReducers` est une fonction fournie par Redux Toolkit qui permet de fusionner plusieurs reducers en un seul reducer composite.

Question 4

Comment peut-on inclure certains reducers de la persistance avec Redux Persist ?

- ☐ En utilisant l'option `includeReducers` lors de la configuration
- ☐ En utilisant l'option `purge` lors de la configuration
- ☒ En utilisant l'option `whitelist` lors de la configuration

-  Pour inclure certains reducers dans la persistance avec Redux Persist, vous pouvez utiliser l'option `whitelist` lors de la configuration. L'option `whitelist` permet de spécifier les reducers qui doivent être inclus dans la persistance, tandis que les autres seront exclus.

Question 5

Quelle commande devez-vous taper dans le terminal pour installer Redux Toolkit ?

- ☐ `npm install react-toolkit`
- ☒ `npm install @reduxjs/toolkit`
- ☐ `npm install redux`

-  Pour installer Redux Toolkit, vous devez taper la commande `npm install @reduxjs/toolkit` dans le terminal. Cette commande installe la dernière version de Redux Toolkit à partir du registre npm.